

SOLUZIONE

Backend REST API con Node.js & Express

Sistema di Gestione Autonoleggio — Codice completo

1. Panoramica

Soluzione completa e funzionante dell'esercizio: un backend REST di sola lettura con 6 endpoint GET, tutti costruiti richiamando funzioni SQL tramite `sequelize.fn` e `sequelize.literal`. Il codice riportato qui è stato verificato eseguendolo contro un'istanza reale di MariaDB.

Stack e comandi essenziali:

- Node.js 20+, Express 4, Sequelize 6, driver mariadb, dotenv
- Installazione dipendenze: `npm install`
- Configurazione: copiare `.env.example` in `.env` e inserire le credenziali del database
- Database: `mysql ... < sql/create_tables.sql` e poi `sql/seed_data.sql`
- Avvio: `npm start` (server su `http://localhost:3000`)

2. Struttura del progetto

```
autonoleggio-soluzione/  
├── app.js                      # entry point Express  
├── config/database.js         # connessione Sequelize + MariaDB  
├── models/  
│   ├── index.js              # modelli + associazioni (M:N + pivot)  
│   ├── Cliente.js  
│   ├── Auto.js  
│   └── Noleggio.js           # tabella pivot con attributi propri  
├── routes/  
│   ├── clienti.js            # endpoint 1, 2, 3  
│   ├── auto.js               # endpoint 4, 5  
│   └── statistiche.js        # endpoint 6  
├── sql/  
│   ├── create_tables.sql  
│   └── seed_data.sql         # 10 clienti, 8 auto, 24 noleggi  
├── postman/autonoleggio.json  
├── .env.example  
├── package.json  
└── README.md
```

3. Punti chiave della soluzione

Tre scelte tecniche meritano attenzione, perché sono i punti in cui è più facile sbagliare:

Aggregati ed elenco nella stessa risorsa (endpoint 5)

COUNT e AVG collasano le righe, mentre l'elenco dei clienti è multi-riga: non possono convivere nella stessa query. La soluzione usa due query separate (una per gli aggregati sulla pivot, una per l'elenco) e le unisce in un unico oggetto JSON.

TIMESTAMPDIFF con include (endpoint 2)

Quando si combina un attributo calcolato con una JOIN (l'include delle auto), la colonna va qualificata con il nome del modello: `sequelize.col('Cliente.dataNascita')`, altrimenti diventa ambigua.

SUM con DATEDIFF e arrotondamento (endpoint 6)

L'incasso è `SUM(DATEDIFF(dataRiconsegna, dataRitiro) * prezzoGiornaliero)` scritto con `sequelize.literal`. Poiché il prezzo è di tipo `FLOAT`, il risultato viene avvolto in `ROUND(..., 2)` per evitare il rumore in virgola mobile.

4. Codice sorgente

4.1 Configurazione

package.json

```
{
  "name": "autonoleggio-rest-api",
  "version": "1.0.0",
  "description": "Backend REST di sola lettura per la gestione di un autonoleggio (Sequelize + sequelize.fn)",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "engines": {
    "node": ">=20"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mariadb": "^3.3.1",
    "sequelize": "^6.37.3"
  }
}
```

.env.example

```
# Rinomina questo file in .env e inserisci le tue credenziali
DB_HOST=localhost
DB_PORT=3306
DB_NAME=autonoleggio
DB_USER=root
DB_PASSWORD=
PORT=3000
```

config/database.js

```
require('dotenv').config();
const { Sequelize } = require('sequelize');

// Connessione unica a MariaDB, riusata da tutti i modelli e le route.
const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT || 3306,
    dialect: 'mariadb',
    logging: false,
  }
);
```

```
// Le tabelle sono create a mano dallo script SQL e non hanno
// le colonne createdAt/updatedAt: le disattiviamo di default.
define: { timestamps: false, freezeTableName: true }
}
);

module.exports = sequelize;
```

4.2 Modelli e associazioni

models/index.js

```
const sequelize = require('../config/database');

const Cliente = require('./Cliente')(sequelize);
const Auto = require('./Auto')(sequelize);
const Noleggio = require('./Noleggio')(sequelize);

// --- Relazione multi-a-molti con attributi sulla tabella pivot ---
// Usando il MODELLO Noleggio come `through`, gli attributi propri della
// pivot (dataRitiro, dataRiconsegna, valutazione) sono accessibili
// nelle query con include.
Cliente.belongsToMany(Auto, { through: Noleggio, foreignKey: 'ClienteId', otherKey: 'AutoId' });
Auto.belongsToMany(Cliente, { through: Noleggio, foreignKey: 'AutoId', otherKey: 'ClienteId' });

// --- Relazioni dirette con la pivot (comode per le query di aggregazione) ---
Cliente.hasMany(Noleggio, { foreignKey: 'ClienteId', as: 'noleggi' });
Auto.hasMany(Noleggio, { foreignKey: 'AutoId', as: 'noleggi' });
Noleggio.belongsTo(Cliente, { foreignKey: 'ClienteId' });
Noleggio.belongsTo(Auto, { foreignKey: 'AutoId' });

module.exports = { sequelize, Cliente, Auto, Noleggio };
```

models/Cliente.js

```
const { DataTypes } = require('sequelize');

// Tabella principale: Cliente
module.exports = (sequelize) => {
  return sequelize.define('Cliente', {
    id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
    nome: { type: DataTypes.STRING, allowNull: false },
    cognome: { type: DataTypes.STRING, allowNull: false },
    email: { type: DataTypes.STRING, allowNull: false, unique: true },
    telefono: { type: DataTypes.STRING },
    dataNascita: { type: DataTypes.DATEONLY, allowNull: false },
    numeroPatente: { type: DataTypes.STRING },
    dataIscrizione: { type: DataTypes.DATE, defaultValue: DataTypes.NOW }
  }, { tableName: 'Cliente' });
};
```

models/Auto.js

```
const { DataTypes } = require('sequelize');

// Tabella secondaria: Auto (multi-a-molti con Cliente)
module.exports = (sequelize) => {
  return sequelize.define('Auto', {
    id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
    targa: { type: DataTypes.STRING, allowNull: false, unique: true },
    marca: { type: DataTypes.STRING, allowNull: false },
    modello: { type: DataTypes.STRING, allowNull: false },
    prezzoGiornaliero: { type: DataTypes.FLOAT, allowNull: false },
    annoImmatricolazione: { type: DataTypes.INTEGER, allowNull: false },
    cilindrata: { type: DataTypes.INTEGER, allowNull: false }
  });
};
```

```
    }, { tableName: 'Auto' });
  };

```

models/Noleggio.js

```
const { DataTypes } = require('sequelize');

// Tabella associativa (pivot) con attributi propri.
// La chiave primaria e' COMPOSTA: (ClienteId, AutoId).
module.exports = (sequelize) => {
  return sequelize.define('Noleggio', {
    ClienteId: { type: DataTypes.INTEGER, primaryKey: true },
    AutoId: { type: DataTypes.INTEGER, primaryKey: true },
    dataRitiro: { type: DataTypes.DATEONLY, allowNull: false },
    dataRiconsegna: { type: DataTypes.DATEONLY, allowNull: false },
    valutazione: { type: DataTypes.INTEGER }
  }, { tableName: 'Noleggio' });
};

```

4.3 Entry point

app.js

```
require('dotenv').config();
const express = require('express');
const { sequelize } = require('./models');

const clientiRouter = require('./routes/clienti');
const autoRouter = require('./routes/auto');
const statisticheRouter = require('./routes/statistiche');

const app = express();
app.use(express.json());

app.use('/api/clienti', clientiRouter);
app.use('/api/auto', autoRouter);
app.use('/api/statistiche', statisticheRouter);

// Endpoint non trovato
app.use((req, res) => {
  res.status(404).json({ error: 'Endpoint non trovato.' });
});

const PORT = process.env.PORT || 3000;

sequelize.authenticate()
  .then(() => {
    console.log('Connessione al database riuscita.');
```

```
    app.listen(PORT, () => console.log(`Server in ascolto su http://localhost:${PORT}`));
  })
  .catch((err) => {
    console.error('Impossibile connettersi al database:', err.message);
    process.exit(1);
  });

```

4.4 Route (i 6 endpoint)

routes/clienti.js

```
const express = require('express');
const { Op } = require('sequelize');
const sequelize = require('../config/database');
const { Cliente, Auto } = require('../models');

const router = express.Router();

```

```
// =====
// 1. GET /api/clienti
//   nomeCompleto -> CONCAT(nome, ' ', cognome) [funzione di stringa]
//   dataIscrizioneFormattata -> DATE_FORMAT(...) [funzione di conversione]
// =====
router.get('/', async (req, res) => {
  try {
    const clienti = await Cliente.findAll({
      attributes: [
        'id', 'email', 'telefono', 'numeroPatente',
        [sequelize.fn('CONCAT', sequelize.col('nome'), ' ', sequelize.col('cognome')),
        'nomeCompleto'],
        [sequelize.fn('DATE_FORMAT', sequelize.col('dataIscrizione'), '%d/%m/%Y'),
        'dataIscrizioneFormattata']
      ],
      order: [['cognome', 'ASC'], ['nome', 'ASC']]
    });
    res.json(clienti);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// =====
// 3. GET /api/clienti/ricerca?q=...
//   DEVE stare PRIMA di /:id, altrimenti 'ricerca' viene letto come id.
//   Ricerca case-insensitive su nome/cognome con Op.like (su MariaDB LIKE
//   e' gia' case-insensitive con collation utf8_general_ci).
// =====
router.get('/ricerca', async (req, res) => {
  try {
    const { q } = req.query;
    if (!q) {
      return res.status(400).json({ error: "Parametro di ricerca 'q' obbligatorio." });
    }
    const clienti = await Cliente.findAll({
      attributes: [
        'id', 'email',
        [sequelize.fn('CONCAT', sequelize.col('nome'), ' ', sequelize.col('cognome')),
        'nomeCompleto']
      ],
      where: {
        [Op.or]: [
          { nome: { [Op.like]: `_${q}%` } },
          { cognome: { [Op.like]: `_${q}%` } }
        ]
      }
    });
    res.json(clienti);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// =====
// 2. GET /api/clienti/:id
//   eta -> TIMESTAMPDIF (YEAR, dataNascita, NOW()) [funzione data]
//   + elenco auto noleggiate con gli attributi della pivot Noleggio.
// =====
router.get('/:id', async (req, res) => {
  try {
    const cliente = await Cliente.findByPk(req.params.id, {
      attributes: {
        include: [
          [
            sequelize.fn('TIMESTAMPDIF', sequelize.literal('YEAR'),
            sequelize.col('Cliente.dataNascita'), sequelize.fn('NOW')),

```

```

        'eta'
      ]
    ]
  },
  include: [{
    model: Auto,
    through: { attributes: ['dataRitiro', 'dataRiconsegna', 'valutazione'] }
  }]
});
if (!cliente) {
  return res.status(404).json({ error: 'Cliente non trovato.' });
}
res.json(cliente);
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

module.exports = router;

```

routes/auto.js

```

const express = require('express');
const sequelize = require('../config/database');
const { Auto, Cliente, Noleggio } = require('../models');

const router = express.Router();

// =====
// 4. GET /api/auto
//   modelloCompleto -> CONCAT(marca, ' ', modello)
//   prezzoFormattato -> CONCAT(FORMAT(prezzoGiornaliero, 2), ' €')
//   cilindrataFormattata -> CONCAT(cilindrata, ' cc')
// =====
router.get('/', async (req, res) => {
  try {
    const auto = await Auto.findAll({
      attributes: [
        'id', 'targa', 'annoImmatricolazione',
        [sequelize.fn('CONCAT', sequelize.col('marca'), ' ', sequelize.col('modello')),
        'modelloCompleto'],
        [sequelize.literal("CONCAT(FORMAT(prezzoGiornaliero, 2), ' €')"), 'prezzoFormattato'],
        [sequelize.literal("CONCAT(cilindrata, ' cc')"), 'cilindrataFormattata']
      ],
      order: [['marca', 'ASC'], ['modello', 'ASC']]
    });
    res.json(auto);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// =====
// 5. GET /api/auto/:id
//   numeroNoleggi -> COUNT
//   valutazioneMedia -> ROUND(AVG(valutazione), 1)
//   + elenco clienti che l'hanno noleggiata.
//   Nota: gli aggregati (COUNT/AVG) collassano le righe, mentre l'elenco
//   clienti e' multi-riga -> si usano due query separate e si uniscono
//   i risultati in un solo oggetto JSON.
// =====
router.get('/:id', async (req, res) => {
  try {
    const auto = await Auto.findByPk(req.params.id, {
      attributes: [
        'id', 'targa', 'marca', 'modello', 'prezzoGiornaliero', 'annoImmatricolazione',
        'cilindrata',

```

```

        [sequelize.fn('CONCAT', sequelize.col('marca'), ' ', sequelize.col('modello')),
        'modelloCompleto']
    ],
    include: [{
        model: Cliente,
        attributes: ['id', 'nome', 'cognome', 'email'],
        through: { attributes: ['dataRitiro', 'dataRiconsegna', 'valutazione'] }
    }]
});

if (!auto) {
    return res.status(404).json({ error: 'Auto non trovata.' });
}

// Query di aggregazione sulla pivot per questa auto.
const stats = await Noleggio.findOne({
    where: { AutoId: req.params.id },
    attributes: [
        [sequelize.fn('COUNT', sequelize.col('AutoId')), 'numeroNoleggi'],
        [sequelize.fn('ROUND', sequelize.fn('AVG', sequelize.col('valutazione')), 1),
        'valutazioneMedia']
    ],
    raw: true
});

const risultato = auto.toJSON();
risultato.numeroNoleggi = Number(stats.numeroNoleggi) || 0;
risultato.valutazioneMedia = stats.valutazioneMedia !== null ? Number(stats.valutazioneMedia)
    : null;

res.json(risultato);
} catch (err) {
    res.status(500).json({ error: err.message });
}
});

module.exports = router;

```

routes/statistiche.js

```

const express = require('express');
const sequelize = require('../config/database');
const { Auto, Noleggio } = require('../models');

const router = express.Router();

// =====
// 6. GET /api/statistiche/auto-piu-noleggiate
// Classifica delle auto per numero di noleggi (decrescente):
//     numeroNoleggi    -> COUNT
//     incassoTotale    -> SUM(DATEDIFF(dataRiconsegna, dataRitiro) * prezzoGiornaliero)
//     valutazioneMedia -> ROUND(AVG(valutazione), 1)
// Query con GROUP BY sull'auto e ORDER BY sul conteggio.
// =====
router.get('/auto-piu-noleggiate', async (req, res) => {
    try {
        const classifica = await Auto.findAll({
            attributes: [
                'id',
                [sequelize.fn('CONCAT', sequelize.col('Auto.marca'), ' ', sequelize.col('Auto.modello')),
                'modelloCompleto'],
                [sequelize.fn('COUNT', sequelize.col('noleggi.AutoId')), 'numeroNoleggi'],
                [sequelize.literal('ROUND(SUM(DATEDIFF(`noleggi`.`dataRiconsegna`, `noleggi`.`dataRitiro`)
                * `Auto`.`prezzoGiornaliero`, 2)'), 'incassoTotale'],
                [sequelize.fn('ROUND', sequelize.fn('AVG', sequelize.col('noleggi.valutazione')), 1),
                'valutazioneMedia']
            ],

```

```

    include: [{ model: Noleggio, as: 'noleggi', attributes: [] }],
    group: ['Auto.id'],
    order: sequelize.literal('numeroNoleggi DESC'),
    subQuery: false,
    raw: true
  });

  // Il driver puo' restituire i numeri come stringhe: li convertiamo.
  const risultato = classifica.map((r) => ({
    modelloCompleto: r.modelloCompleto,
    numeroNoleggi:    Number(r.numeroNoleggi),
    incassoTotale:    Number(r.incassoTotale),
    valutazioneMedia: r.valutazioneMedia !== null ? Number(r.valutazioneMedia) : null
  }));

  res.json(risultato);
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

module.exports = router;

```

4.5 Script SQL

sql/create_tables.sql

```

-- =====
-- Autonoleggio - Creazione tabelle (MariaDB)
-- Ordine: Cliente e Auto (senza FK) -> Noleggio (FK verso entrambe)
-- =====

-- Tabella principale: Cliente
CREATE TABLE IF NOT EXISTS Cliente (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nome        VARCHAR(255) NOT NULL,
  cognome     VARCHAR(255) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  telefono    VARCHAR(50),
  dataNascita DATE NOT NULL,
  numeroPatente VARCHAR(50),
  dataIscrizione DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Tabella secondaria: Auto (multi-a-molti con Cliente)
CREATE TABLE IF NOT EXISTS Auto (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  targa       VARCHAR(20) NOT NULL UNIQUE,
  marca       VARCHAR(255) NOT NULL,
  modello     VARCHAR(255) NOT NULL,
  prezzoGiornaliero FLOAT NOT NULL,
  annoImmatricolazione INT NOT NULL,
  cilindrata  INT NOT NULL
);

-- Tabella associativa: Noleggio (PK composta + attributi propri)
CREATE TABLE IF NOT EXISTS Noleggio (
  ClienteId   INT NOT NULL,
  AutoId      INT NOT NULL,
  dataRitiro  DATE NOT NULL,
  dataRiconsegna DATE NOT NULL,
  valutazione INT,
  PRIMARY KEY (ClienteId, AutoId),
  FOREIGN KEY (ClienteId) REFERENCES Cliente(id),
  FOREIGN KEY (AutoId)   REFERENCES Auto(id)
);

```

sql/seed_data.sql

```
-- =====
-- Autonoleggio - Dati di test
-- 10 clienti, 8 auto, 24 noleggi (coppie Cliente-Auto uniche).
-- =====

INSERT INTO Cliente (nome, cognome, email, telefono, dataNascita, numeroPatente, dataIscrizione)
VALUES
('Mario', 'Rossi', 'mario.rossi@email.it', '3401111111', '1985-03-12', 'IT12345',
'2024-01-15 10:00:00'),
('Laura', 'Bianchi', 'laura.bianchi@email.it', '3402222222', '1990-07-22', 'IT23456',
'2024-02-03 11:30:00'),
('Giuseppe', 'Verdi', 'giuseppe.verdi@email.it', '3403333333', '1978-11-05', 'IT34567',
'2024-02-20 09:15:00'),
('Anna', 'Esposito', 'anna.esposito@email.it', '3404444444', '1995-01-30', 'IT45678',
'2024-03-10 14:45:00'),
('Marco', 'Ferrari', 'marco.ferrari@email.it', '3405555555', '1982-09-18', 'IT56789',
'2024-03-25 16:20:00'),
('Sara', 'Russo', 'sara.russo@email.it', '3406666666', '1998-05-14', 'IT67890',
'2024-04-05 08:50:00'),
('Luca', 'Romano', 'luca.romano@email.it', '3407777777', '1975-12-01', 'IT78901',
'2024-04-18 12:10:00'),
('Giulia', 'Costa', 'giulia.costa@email.it', '3408888888', '1993-08-27', 'IT89012',
'2024-05-02 15:00:00'),
('Paolo', 'Greco', 'paolo.greco@email.it', '3409999999', '1988-06-09', 'IT90123',
'2024-05-20 10:40:00'),
('Chiara', 'Conti', 'chiara.conti@email.it', '3400000000', '2000-02-28', 'IT01234',
'2024-06-01 13:25:00');

INSERT INTO Auto (targa, marca, modello, prezzoGiornaliero, annoImmatricolazione, cilindrata)
VALUES
('AA123BB', 'Fiat', 'Panda', 29.90, 2021, 1200),
('CC456DD', 'Volkswagen', 'Golf', 49.50, 2020, 1600),
('EE789FF', 'Toyota', 'Yaris', 34.90, 2022, 1500),
('GG012HH', 'Audi', 'A3', 69.90, 2021, 2000),
('II345LL', 'Renault', 'Clio', 32.50, 2019, 1200),
('MM678NN', 'BMW', 'Serie 1', 74.90, 2022, 2000),
('OO901PP', 'Ford', 'Focus', 44.90, 2020, 1500),
('QQ234RR', 'Mercedes-Benz', 'Classe A', 79.90, 2023, 1600);

-- Noleggi: ogni coppia (ClienteId, AutoId) e' unica (PK composta).
INSERT INTO Noleggio (ClienteId, AutoId, dataRitiro, dataRiconsegna, valutazione) VALUES
-- Auto 1 (Fiat Panda) - molto noleggiata
(1, 1, '2025-01-10', '2025-01-15', 5),
(2, 1, '2025-02-01', '2025-02-04', 4),
(3, 1, '2025-03-05', '2025-03-12', 5),
(4, 1, '2025-04-20', '2025-04-22', 3),
(5, 1, '2025-05-15', '2025-05-25', 4),
-- Auto 2 (VW Golf)
(1, 2, '2025-01-20', '2025-01-27', 4),
(2, 2, '2025-03-10', '2025-03-13', 5),
(6, 2, '2025-06-01', '2025-06-08', 3),
-- Auto 3 (Toyota Yaris) - la piu' noleggiata
(3, 3, '2025-02-14', '2025-02-16', 5),
(4, 3, '2025-03-01', '2025-03-05', 4),
(5, 3, '2025-04-10', '2025-04-18', 5),
(6, 3, '2025-05-05', '2025-05-07', 4),
(7, 3, '2025-06-12', '2025-06-20', 5),
(8, 3, '2025-07-01', '2025-07-04', 3),
-- Auto 4 (Audi A3)
(7, 4, '2025-03-15', '2025-03-20', 5),
(8, 4, '2025-05-22', '2025-05-24', 4),
-- Auto 5 (Renault Clio)
(9, 5, '2025-02-05', '2025-02-12', 3),
(10, 5, '2025-04-01', '2025-04-03', 4),
(1, 5, '2025-06-15', '2025-06-22', 5),
-- Auto 6 (BMW Serie 1)

```

```
(2, 6, '2025-03-25', '2025-03-28', 5),
(9, 6, '2025-07-10', '2025-07-15', 4),
-- Auto 7 (Ford Focus)
(10, 7, '2025-05-10', '2025-05-14', 4),
(3, 7, '2025-06-25', '2025-06-30', NULL),
-- Auto 8 (Mercedes Classe A)
(4, 8, '2025-07-20', '2025-07-25', 5);
```

5. Esempi di risposta

Output reale prodotto dalla soluzione con i dati di test (estratti).

Esempio — endpoint 2

```
GET /api/clienti/1
{
  "id": 1,
  "nome": "Mario",
  "cognome": "Rossi",
  "email": "mario.rossi@email.it",
  "dataNascita": "1985-03-12",
  "dataIscrizione": "2024-01-15T10:00:00.000Z",
  "eta": 41,
  "Autos": [
    {
      "marca": "Fiat", "modello": "Panda", "prezzoGiornaliero": 29.9,
      "Noleggio": {
        "dataRitiro": "2025-01-10",
        "dataRiconsegna": "2025-01-15",
        "valutazione": 5
      }
    }
  ]
}
```

Esempio — endpoint 6

```
GET /api/statistiche/auto-piu-noleggiate
[
  { "modelloCompleto": "Toyota Yaris", "numeroNoleggi": 6, "incassoTotale": 942.3,
    "valutazioneMedia": 4.3 },
  { "modelloCompleto": "Fiat Panda", "numeroNoleggi": 5, "incassoTotale": 807.3,
    "valutazioneMedia": 4.2 },
  { "modelloCompleto": "Volkswagen Golf", "numeroNoleggi": 3, "incassoTotale": 841.5,
    "valutazioneMedia": 4 },
  { "modelloCompleto": "Renault Clio", "numeroNoleggi": 3, "incassoTotale": 520,
    "valutazioneMedia": 4 }
]
```